What Changed in the Room?

A guide to the project, assuming nothing

Ramdas Narayan Chaugale

MSc Artificial Intelligence
Queen's University Belfast

This project takes two phone videos of the same room, filmed at different times, and works out what moved, what appeared and what is gone, in three dimensions.

These notes are written so that someone with no technical background can follow what it does and why. Chapter 1 stands on its own. Each chapter after that assumes a little more, and every tool is explained in plain words before it is used.

1 what it is, with no jargon at all. **2** why it exists, and why a robot would want it. **3** every tool, what it does here, and why it was chosen over the alternatives. **4** what actually happens, step by step. **5** the decisions that mattered. **6** the fake room used to check the answers. **7** a glossary. **8** the questions worth being ready for, including what went wrong.

The hard part of this problem is not spotting that something is different. It is being sure that an empty space means the object left, rather than that nobody looked there.

October 16, 2026

Contents

1 What this is, in plain words **2**
- 1.1 Why that is harder than it sounds . . . 2
- 1.2 What goes in . . . 2
- 1.3 What comes out . . . 2

2 Why it exists **4**
- 2.1 Why a robot would want this . . . 4
- 2.2 Why it is not solved already . . . 4

3 Every tool, and why **5**
- 3.1 COLMAP, used through pycolmap . . . 5
- 3.2 Depth Anything V2 . . . 5
- 3.3 CLIP . . . 6
- 3.4 DBSCAN, from scikit-learn . . . 6
- 3.5 OpenCV . . . 6
- 3.6 Plotly . . . 6
- 3.7 Matplotlib . . . 7
- 3.8 NumPy . . . 7

4 What actually happens, step by step **8**
- 4.1 Turn the videos into pictures . . . 8
- 4.2 Work out the shape of the room, from both videos at once . . . 8
- 4.3 Fill in the depth, and fix the scale . . . 8
- 4.4 Record what each visit knows, including what was empty . . . 9
- 4.5 Compare . . . 9
- 4.6 Group, name, and pair up . . . 9

5 The decisions that mattered **11**
- 5.1 Reconstructing both visits together . . . 11
- 5.2 Tracking empty space, not just full space . . . 11
- 5.3 Refusing to answer when the evidence is missing . . . 11
- 5.4 Refusing to name things it cannot see properly . . . 11
- 5.5 Testing against a known answer . . . 11

6 The test room **13**
- 6.1 What the test found . . . 13
- 6.2 Why the test room looks so strange . . . 13

7 Glossary **14**

8 Questions worth being ready for **15**
- 8.1 Why not just compare the two point clouds? . . . 15
- 8.2 Why one reconstruction instead of aligning two? . . . 15
- 8.3 How do you know it works? . . . 15
- 8.4 What are the weaknesses? . . . 15
- 8.5 What would you do next? . . . 15
- 8.6 What did you get wrong along the way? . . . 15

Chapter 1

What this is, in plain words

Imagine you film a short video walking around your living room on Monday. On Friday somebody moves the armchair, takes away a cardboard box, and leaves a rucksack by the door. You film the room again.

This project takes those two videos and tells you:

- the armchair moved, and roughly how far
- the box is gone
- a rucksack appeared
- the table did not move

It also shows this as a three dimensional model you can spin round in a web browser, with the things that changed picked out in colour. Nobody has to tell it what to look for. It works out the shape of the room from the videos themselves.

1.1 Why that is harder than it sounds

If you simply compared the two videos frame by frame, almost everything would look different. You stood in a different spot. You held the phone at a different angle. You walked a slightly different route. The light changed.

The genuinely hard part is not spotting that something is different. It is being certain that an empty space means **the object was taken away** rather than **I just did not point the camera there the second time**.

Watch out

A person does this without thinking. A computer has to be told the difference, and most systems get it wrong: they report everything the camera missed as a change. On real footage, where nobody walks the identical route twice, that is most of the room.

That problem is what the whole project is built around, and Chapter 4 explains the two ideas that solve it.

1.2 What goes in

Two ordinary phone videos of the same room, twenty to sixty seconds each, walked along a similar route. Nothing else. No special camera, no depth sensor, no markers placed in the room, no measurements taken by hand.

1.3 What comes out

Three things.

- **An interactive 3D scene** in a web browser. The unchanged room is grey, removals are red, additions are green, moves are amber. You can drag to rotate it.
- **A written summary** in ordinary English, for example: “A chair moved. It was in the centre on the left side and is now towards the back on the right side, about 2.4 m away.”
- **A data file** listing every change with its position, size, name and the evidence behind the naming, for anything that needs to read the result automatically.

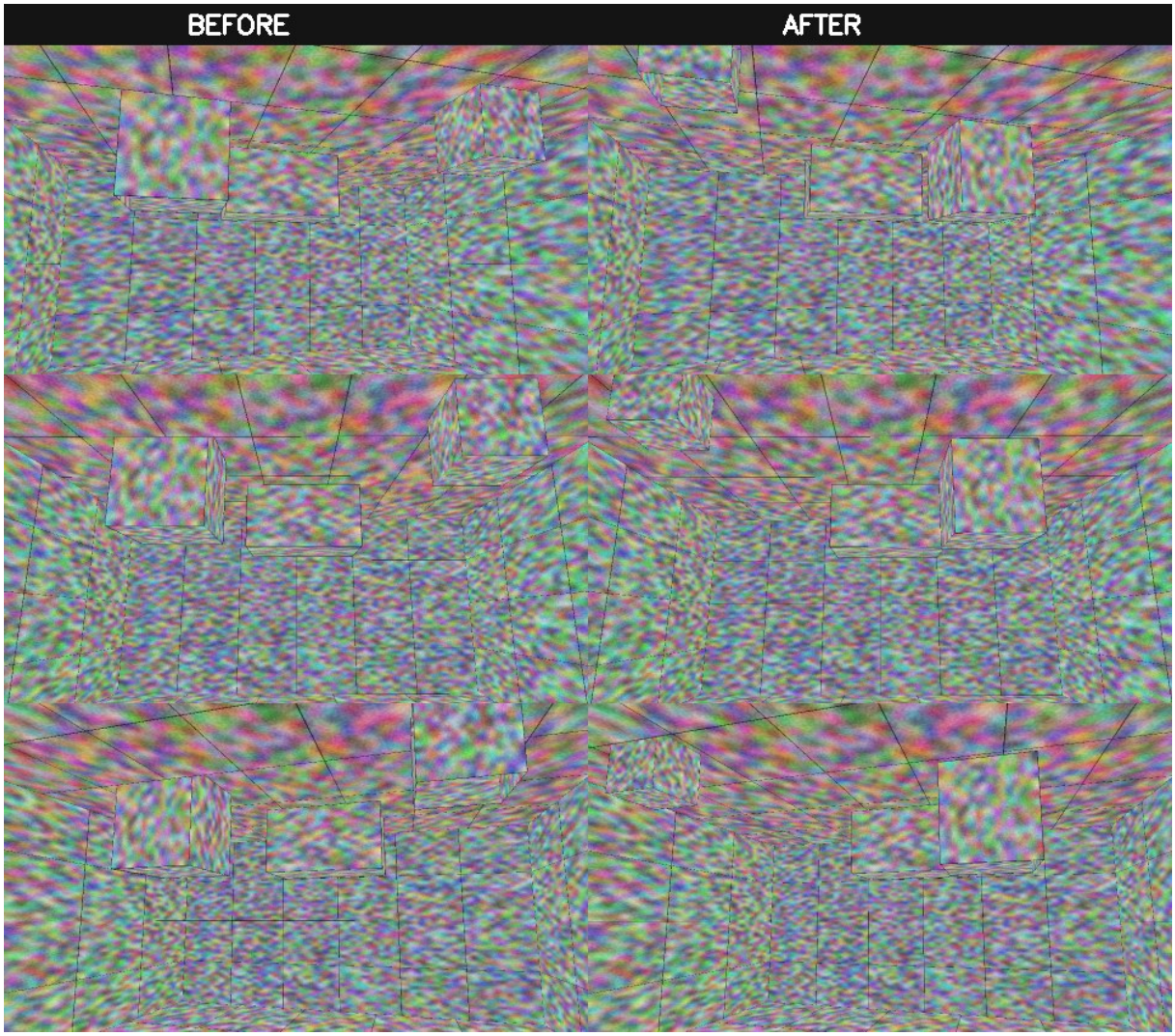


Figure 1.1: Three matching moments from the two recordings used to test the system, before on the left and after on the right. This is a drawn test room rather than a real one, for reasons explained in Chapter 6.

Chapter 2

Why it exists

It was built as a technical challenge for a job application to Humanoid, a London company building humanoid robots. Their brief was to compare two phone recordings of a room and identify meaningful changes in three dimensions, while distinguishing genuine change from differences in camera viewpoint, occlusion and incomplete observation.

That last clause is the whole exercise. Anyone can write something that notices two pictures are not identical. The brief asks for something that knows when it is entitled to say so.

2.1 Why a robot would want this

A robot working in a warehouse, a hospital or a home builds a map of its surroundings. The world then moves. Pallets get shifted. Doors close. Boxes appear in corridors. Furniture is rearranged around it. The robot has to notice, without being re-shown the whole building every morning.

The consequence of being wrong is not symmetrical.
If the robot wrongly believes a chair is still there, it goes around a chair that is not there. It wastes a few seconds.
If the robot wrongly believes a corridor is clear, because it never actually looked, it drives into something.

So a system like this should prefer saying *I do not know* over guessing. That single principle runs through every design decision in the project, and it is why the system is deliberately built to under-report rather than over-report.

2.2 Why it is not solved already

Depth cameras and laser scanners make this much easier, and plenty of robots carry them. This project deliberately uses nothing but an ordinary phone camera, because the brief said phone recordings.

That removes the one thing that would make the problem simple, which is a direct measurement of distance. Everything about distance here has to be inferred from how the view changes as the camera moves, and from a model that has learned what near and far look like. Chapter 3 covers both.

Chapter 3

Every tool, and why

Each tool gets a plain explanation, then what it does in this project, then why it was chosen over the alternatives.

3.1 COLMAP, used through pycolmap

What it is. Software that works out the three dimensional shape of a scene from ordinary two dimensional photographs, and where the camera was standing for each one. The technique is called *structure from motion*. It is the same idea your eyes use: two slightly different views of the same thing let you judge depth.

In this project

It takes every frame from both videos and returns two things. First, the position and angle of the camera for each frame. Second, a sparse cloud of three dimensional points marking corners and edges it managed to recognise in several frames at once.

Why this one. It is the standard tool for the job and it is free. The Python package `pycolmap` installs with pip, so there is no separate program to set up, which matters when somebody else has to run your work. A robotics team will recognise it immediately.

3.2 Depth Anything V2

What it is. A neural network that looks at a single photograph and estimates how far away every pixel is. It was trained on an enormous number of images, so it has learned the cues that make things look near or far: perspective, relative size, and how objects overlap each other.

In this project

COLMAP returns a few hundred accurate points per frame, which is nowhere near enough to say whether a chair is standing in a corner. This fills in a depth estimate for every single pixel.

Why this one. It runs on a laptop with no dedicated graphics card. The textbook alternative, dense multi-view stereo, is more accurate but effectively requires a powerful GPU, which would have made the project impossible to run or review on an ordinary machine.

Watch out

Its output has no sense of scale. It knows the sofa is nearer than the wall; it does not know the sofa is two metres away. Section 4.3 explains how that is fixed, and it is one of the more interesting parts of the project.

3.3 CLIP

What it is. A model trained on hundreds of millions of image and caption pairs. Give it a picture and a list of candidate descriptions, and it scores how well each description fits. It was never trained on a fixed list of objects, so the list can be changed without retraining anything.

In this project

Once the geometry has found a lump of space that changed, CLIP is shown a cropped photograph of that space and picks the best match from a list of household objects, so the final report says “a chair” instead of “cluster 3”.

Why this one. The geometry has already decided *where* the object is, so a full object detector is not needed, only a name. CLIP is small, runs on a laptop, and needs no training data.

3.4 DBSCAN, from scikit-learn

What it is. An algorithm that finds groups in scattered data. Unlike most clustering methods, you do not tell it how many groups to expect. It finds however many there are, and labels loose scattered points as noise rather than forcing them into a group.

In this project

After the comparison there are thousands of individual cells of space marked as changed. DBSCAN groups touching cells into objects, turning “four thousand changed cells” into “a chair sized lump here and a box sized lump over there”.

Why this one. Not knowing the number of objects in advance is the entire point, which rules out anything that needs that number up front. Its willingness to discard scattered points as noise is exactly what is wanted, because measurement error produces scattered points.

3.5 OpenCV

What it is. The standard library for working with images and video in code.

In this project

Reads the videos, pulls out frames, resizes them, and measures how blurry each one is so the bad ones can be discarded. It also draws the synthetic test room described in Chapter 6.

3.6 Plotly

What it is. A charting library that produces interactive graphics which open in a web browser.

In this project

Produces `scene.html`, the rotatable three dimensional model of the result.

Why this one. The output is a single file that opens in any browser. The usual alternative for 3D work, Open3D, opens a desktop window instead, which is no use to somebody reviewing the work from a link.

3.7 Matplotlib

What it is. The standard Python library for charts.

In this project

Draws the figures used in this document and in the project's README. They are generated from the real output of a real run rather than drawn by hand, so they cannot quietly drift away from what the code actually produces.

3.8 NumPy

What it is. The library that lets Python do arithmetic on millions of numbers quickly.

In this project

Everything numerical. The free space calculation described in Section 4.4 handles tens of millions of points. In ordinary Python that would take hours. In NumPy it takes seconds.

Chapter 4

What actually happens, step by step

4.1 Turn the videos into pictures

Sample a couple of frames per second, shrink them, and throw away the blurry ones. Blur is measured by how sharp the edges are in the picture.

A blurry frame is worse than no frame at all, because the next stage works by recognising the same corner in several different pictures, and blur destroys corners.

4.2 Work out the shape of the room, from both videos at once

Watch out

This is the single most important decision in the project. If only one thing is remembered from these notes, it should be this section.

The obvious approach is to build a three dimensional model from video A, build a second one from video B, and then line the two up.

That approach asks the hardest question first. Lining up two models works by finding the position where they best agree, and it is least reliable exactly when part of the scene has genuinely moved. The things that changed drag the alignment off, and once the alignment is wrong, everything looks changed.

So instead, every frame from **both** videos goes into **one** reconstruction. The walls, the floor and the furniture that stayed put appear in both recordings, so the software matches them across the two videos by itself, and everything emerges in a single shared coordinate system. There is no lining up step to get wrong.

Put simply: **the parts of the room that did not change are what tie the two visits together**. The stable scene is the evidence. The changes are the thing being measured. Doing it the other way round uses the unknown to establish the evidence.

The code refuses to continue if both videos did not end up in the same model. A model containing only video A would report the whole of video B as new, and that would look like a result rather than a failure.

4.3 Fill in the depth, and fix the scale

The points from the previous step are accurate but sparse. Depth Anything fills in every pixel, but its output has no scale.

The fix uses one against the other. Both describe the same frame, so finding the stretch and shift that makes the estimated depths line up with the accurate sparse points converts the whole dense estimate into the same units as the reconstruction. One small calculation per frame, and the dense depth inherits whatever scale the reconstruction has.

Frames with too few sparse points to do this reliably are skipped rather than guessed at.

Watch out

A wrongly scaled depth map does not produce a slightly wrong answer. It produces a confident answer in completely the wrong part of the room, which is far more damaging than no answer.

There is one further wrinkle. Reconstruction from photographs alone can never recover true real-world size, because a doll's house photographed close up looks identical to a real house photographed from further away. So the whole scene is rescaled once, so that typical distances come out room sized. This is not true measurement, but it makes every setting in the configuration file mean roughly what it appears to mean.

4.4 Record what each visit knows, including what was empty

Space is divided into a grid of cells, each five centimetres across. For each visit, every cell is marked as one of three things.

Label	Meaning
occupied	something was measured here
free	a camera ray passed through here on its way to hitting something further away, so this space was looked at and was empty
unknown	neither, so nothing can be said about it

Free is the state that makes this work. Finding it means tracing the straight line from the camera out to each thing it saw, and marking everything along the way as confirmed empty. It is the most expensive part of the whole program and it is the reason the results can be trusted.

The tracing stops slightly short of the surface. Without that margin, the front face of every object would erase itself and solid things would dissolve.

4.5 Compare

Two rules, and the whole argument of the project sits in them.

```
REMOVED   if occupied in visit A  AND  confirmed empty in visit B
ADDED     if occupied in visit B  AND  confirmed empty in visit A
```

Both require positive evidence from the other visit. A cell that held something in A and was simply never looked at in B produces nothing at all, because nothing can honestly be claimed about it.

That one restriction is the difference between a system that reports *what changed* and one that reports *where the camera went*.

4.6 Group, name, and pair up

The changed cells are grouped into objects. Each group is then traced back into the photographs that saw it, cropped out, and shown to CLIP for a name.

One subtle detail matters here. A removed object is cropped from a **visit A** photograph, because by visit B it is not there any more.



Figure 4.1: Why the third state matters. The chair is reported as removed because visit B positively saw through that space. The region behind the pillar is not reported at all, because neither visit could see it. A system that only tracked occupied space would report both as changes.

Watch out

Getting that backwards returns a picture of whatever is sitting in that space now, labels it confidently, and produces output that is wrong while looking entirely reasonable. Those are the worst kind of bugs, because nothing about the output announces the error.

Finally, a disappearance in one place and an appearance in another are reported as a single **move** only if they agree on name, are a similar size, and are not too far apart. Otherwise they stay as two separate events, because that is the honest reading of the evidence.

Chapter 5

The decisions that mattered

5.1 Reconstructing both visits together

Covered in Section 4.2. The single most important choice in the project, and the one worth being able to defend in detail.

5.2 Tracking empty space, not just full space

The thing that stops occlusion being reported as change. Without it the system would be a difference detector rather than a change detector, and on real footage the difference between those two is most of the room.

5.3 Refusing to answer when the evidence is missing

The system will miss real changes in parts of the room that only one visit covered. That is intended, not an oversight.

For a robot, a missed change is a wasted detour. A hallucinated clear path is a collision. Given a choice between the two failure modes, this one is correct.

5.4 Refusing to name things it cannot see properly

An early version of the report confidently announced that a television had moved. It had not. There was no television. CLIP was guessing, at a confidence barely distinguishable from random chance, and the program was printing those guesses as fact.

With twenty nine candidate labels, a model with no idea scores about $1/29 = 0.034$ for its best guess. The actual scores coming back were 0.035. The model was not choosing, it was shrugging.

There is now a floor. Anything scoring near chance is reported as “unidentified object”, and moves are not paired on a name nobody believes. The output went from six invented findings to ten honest ones.

Watch out

This is worth noticing as a general lesson. The system was not broken. It was producing output that looked exactly like a working system, and only checking the confidence numbers revealed that the labels were noise.

5.5 Testing against a known answer

Rather than filming a room and eyeballing the output, the project includes a program that draws a fake room where the changes are scripted in advance. That makes the results checkable rather than merely plausible.

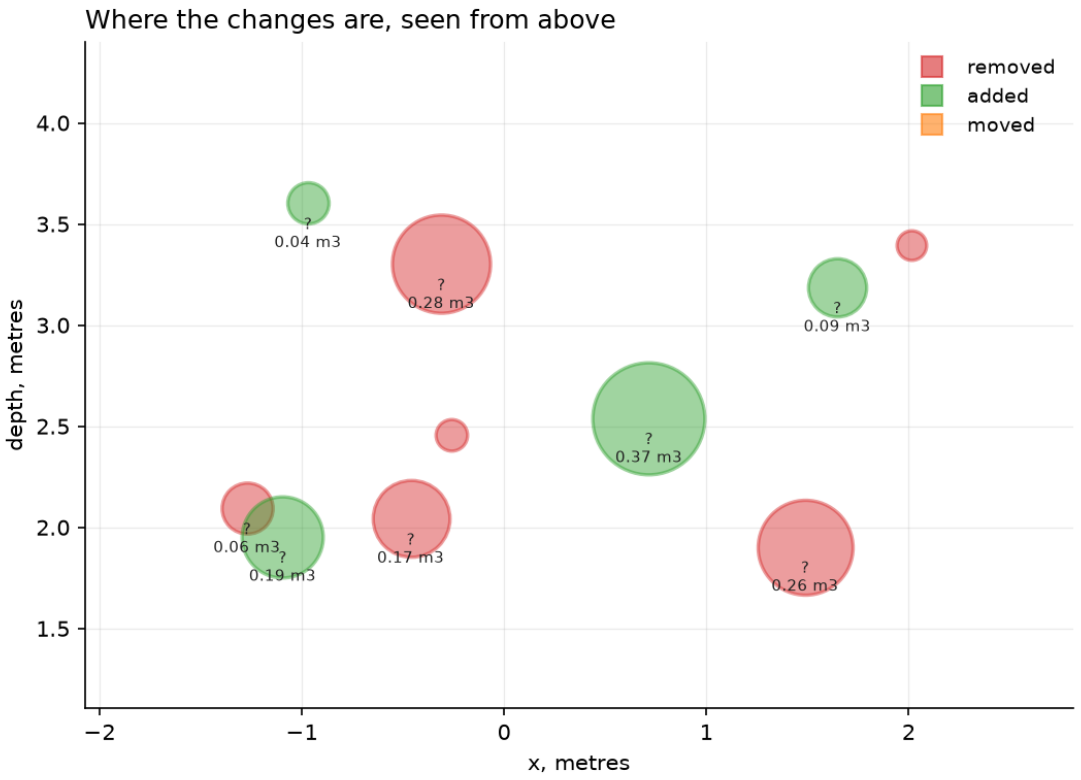


Figure 5.1: The detected changes seen from above, circle size proportional to volume. The four real changes land in four different parts of the room, which is what a result tracking real geometry looks like. A result driven by camera motion would pile up in one corner.

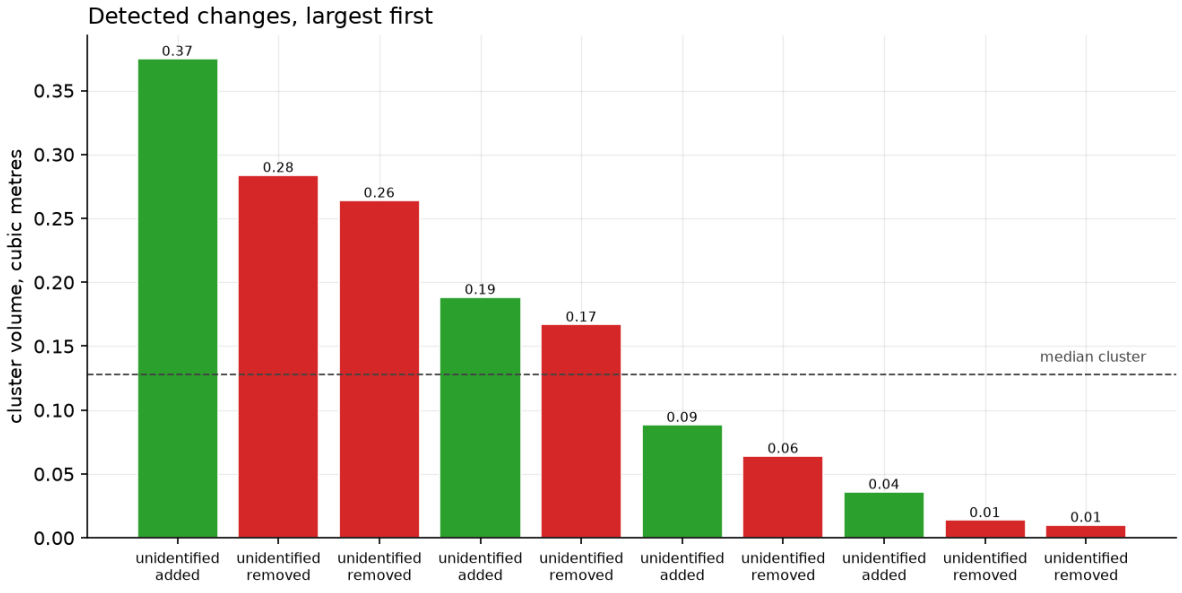


Figure 5.2: Every detected cluster by volume. The four real changes are the four largest. Everything below the median line is depth error on flat walls rather than an object, and it is why a minimum cluster size exists in the configuration.

Chapter 6

The test room

`tools/make_synthetic_room.py` draws two videos of an imaginary room from a moving viewpoint. Between the two, one box moves across the room, one is taken away, one is added, and one stays exactly where it was.

Because the answer is known in advance, the output can be marked rather than admired.

6.1 What the test found

Truth	Found	Evidence
chair moved left to right	yes	0.28 m^3 removed, 0.09 m^3 added
box removed from the right	yes	0.26 m^3 removed, front right
bag added on the left	yes	0.19 m^3 added, front left
table unchanged	yes, correctly silent	not reported

The volumes are sensible rather than arbitrary. A hollow cube of roughly 0.8 m^3 has about 5 m^2 of surface, and at a five centimetre cell size that is about 0.25 m^3 of occupied cells, which is what came back.

Six smaller false positives also appear, the largest 0.37 m^3. They come from depth error on large flat walls, where the depth model has little to anchor on. They are the honest cost of using monocular depth rather than a depth sensor, and they are listed in the project's README rather than hidden.

6.2 Why the test room looks so strange

The room is covered in colourful noise rather than looking realistic, and that is deliberate.

Reconstruction works by recognising the same distinctive patch of texture in several frames. A smooth, evenly coloured surface gives it nothing to grip on. A realistic looking render would reconstruct badly for reasons that have nothing to do with the system being tested, and the test would be measuring the renderer rather than the pipeline.

Watch out

The honest limitation is that CLIP cannot name a cube covered in coloured noise, so every label on the synthetic run comes back as "unidentified object".
The fake room tests the geometry. Only a real room can test the naming. Saying so plainly is better than quietly presenting synthetic labels as though they meant something.

Chapter 7

Glossary

Term	What it means
Structure from motion	Working out three dimensional shape, and where the camera was, from a set of ordinary 2D photographs
Point cloud	A 3D shape stored as a lot of individual points floating in space
Sparse and dense	Sparse means a few hundred accurate points per frame. Dense means a depth value for every pixel
Voxel	A cube shaped cell of 3D space, the three dimensional equivalent of a pixel. Five centimetres across here
Occupancy map	A record of which cells of space contain something, are known to be empty, or were never seen
Ray casting	Tracing the straight line from the camera out through a pixel, to find what the camera looked through on the way
Occlusion	One object hiding another from the camera
Monocular depth	Estimating distance from a single image, rather than from two views
Affine-invariant depth	Depth that is correct in relative terms but has unknown scale and offset. It knows nearer and further, not how far
Zero-shot classification	Naming something using a model that was never specifically trained on that list of names
Clustering	Grouping scattered measurements into objects
SIFT keypoint	A distinctive small patch of an image that can be recognised again from a different angle or distance
Pose	Where the camera was and which way it was pointing
Least squares	A standard way of finding the line that best fits a set of measurements
Softmax	Turning a set of raw scores into numbers that behave like probabilities and add up to one

Chapter 8

Questions worth being ready for

8.1 Why not just compare the two point clouds?

Because absence in a point cloud is ambiguous. It can mean the object left, or it can mean the camera never looked there. On real footage, where nobody walks the identical route twice, the second case accounts for most of the room. Tracking confirmed empty space separates the two.

8.2 Why one reconstruction instead of aligning two?

Aligning two models means solving for the transform that makes them agree, and that is least reliable exactly when part of the scene has moved, which is the only case this project cares about. Doing it jointly lets the unchanged scene do the aligning, which is the right way round.

8.3 How do you know it works?

A synthetic room with changes scripted in advance. All four are found, in the right places, at volumes consistent with the surface area of the objects. The figures in Chapter 5 are generated from that run by code reading the output file, not drawn by hand.

8.4 What are the weaknesses?

Four, stated plainly.

- Monocular depth is the weakest link, and its errors are smooth and plausible, which is the worst kind of error because nothing looks obviously wrong.
- There is no true metric scale, so volumes and distances are consistent but not in real metres.
- Six false positives appear on the test scene, from depth error on flat walls.
- The labelling has only been exercised on noise textured cubes, so it is untested on real objects.

8.5 What would you do next?

Real metric scale, from a known size marker in shot or from the phone's own motion tracking. Dense stereo where a GPU is available. Log-odds occupancy, as used in OctoMap, so confidence accumulates across frames instead of a hard threshold on how many times a cell was hit. An open-vocabulary detector so objects outside the configured list can still be named.

8.6 What did you get wrong along the way?

Three things, all caught by testing rather than by reading the code.

1. **The depth limit was written in metres while the reconstruction used arbitrary units.** The scene reconstructed at roughly twenty eight units deep, the limit was eight, so every single measurement was discarded as too far away and nothing fused at all. Fixed by rescaling the reconstruction once to room sized units.
2. **The first synthetic room had no walls.** Any surface with a single corner behind the camera was being discarded whole, which removed the floor and every wall and left a black void. Fixed by cutting the large surfaces into tiles so the test applies locally.

3. **The naming stage reported guesses as fact,** announcing that a television had moved when the model was scoring at chance level. Fixed with a confidence floor.

All three were invisible from reading the code and obvious from running it against a known answer. That is the argument for building the synthetic test room before trusting anything.